AN ENSEMBLE OF LIGHTWEIGHT VISION TRANSFORMERS FOR EFFICIENT MALWARE DETECTION

A PROJECT REPORT

Submitted by

JOSE MUGILAN D (2116231901016)

LAKSHMIKANTH K (2116231901026)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI 600 025

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this Project report titled **"AN ENSEMBLE OF LIGHTWEIGHT VISION TRANSFORMERS FOR EFFICENT MALWARE DETECTION"** is the bonafide work of **"JOSE MUGILAN D (2116231901016), LAKSHMIKANTH K (2116231901026)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

SIGNATURE

Benedict JN

SUPERVISOR

ASSISTANT PROFESSOR(SG)

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 06/10/2026

Internal Examiner

External Examiner

ABSTRACT

The rapid growth in malware attacks presents a major challenge to computer systems, networks, and digital infrastructure. Traditional signature-based malware detection techniques are effective for known threats but may face difficulties in identifying previously unseen or modified malware variants. Deep learning techniques provide an alternative by automatically learning patterns from malware data. This project proposes an ensemble of lightweight Vision Transformer architectures, MobileViT and TinyViT, for malware-family classification using grayscale malware images. In image-based malware classification, malware samples are represented as images containing visual patterns associated with their byte distributions and file structures. These images are preprocessed and supplied to lightweight deep learning models for feature extraction and classification. MobileViT combines convolutional processing with Transformer-based attention to capture local and global image information. TinyViT provides another compact Vision Transformer architecture for image representation learning. The proposed ensemble combines the class-probability outputs of both models using a probability-fusion method to generate a final malware-family prediction. The project is developed using Python, PyTorch, and image-processing libraries. The Malimg malware image dataset is used to develop and evaluate the image-classification component. The current MobileViT implementation was evaluated on 957 test images belonging to 25 malware families and achieved an accuracy of 98.54%. The reported macro precision, recall, and F1-score were 0.95, 0.95, and 0.95, respectively. These results provide a baseline for the planned TinyViT and ensemble experiments. The final system will compare MobileViT, TinyViT, and the combined ensemble using accuracy, precision, recall, F1-score, parameter count, model size, and inference time. A prototype application may be developed to demonstrate static file analysis. The project also investigates the relationship between classification performance and computational efficiency. The work aims to evaluate whether an ensemble of lightweight Vision Transformers can provide effective malware-family classification while maintaining manageable computational requirements.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Benedict JN**, **Assistant Professor**, for his encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

JOSE MUGILAN D 2116231901026

LAKSHMIKANTH K 2116231901026

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	3
	1.3 OBJECTIVES	3
	1.4 SCOPE	4
	1.5 SECURITY REQUIREMENTS AND SECURITY CONSIDERATIONS	5
	1.6 SDG ALIGNMENT	6
2	**LITERATURE REVIEW**	**7**
	2.1 EXISTING SYSTEMS WITH SECURITY APPROACHES	7
	2.1.1 SIGNATURE-BASED MALWARE DETECTION	7
	2.1.2 STATIC MALWARE ANALYSIS	7
	2.1.3 MACHINE LEARNING-BASED DETECTION	8
	2.1.4 CNN-BASED MALWARE IMAGE CLASSIFICATION	8
	2.1.5 VISION TRANSFORMER-BASED CLASSIFICATION	8
	2.2 RESEARCH GAPS	9
	2.3 PROPOSED SYSTEM	9
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	9
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	10
	2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM	10
	2.3.4 LIMITATIONS OF THE PROPOSED SYSTEM	10
3	**SYSTEM DESIGN**	**11**
	3.1 SYSTEM ARCHITECTURE	11
	3.2 MODULE DESIGN	12
	3.3 DATA FLOW AND UML DIAGRAMS	15
	3.3.1 USE CASE DIAGRAM	15
	3.3.2 ACTIVITY DIAGRAM	16
	3.3.3 SEQUENCE DIAGRAM	16

3.3.4 DATA FLOW DIAGRAM 17
3.4 THREAT MODEL AND ATTACK SURFACE 17
4 CONCLUSION AND FUTURE WORK 26
4.1 CONCLUSION 26
3.5 FUTURE WSECURITY CONTROLS 18
5 SYSTEM DESCRIPTION 19
5.1 DATA COLLECTION 19
5.2 DATA PREPROCESSING 20
5.2.1 ALGORITHMS AND METHODOLOGY 21
5.2.2 RESULTS AND DISCUSSION 22
5.3 ORK 26
REFERENCES 28
APPENDIX 29

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
3.1	SECURITY CONTROLS AGAINST IDENTIFIED THREATS	18
4.1	MOBILEVIT EVALUATION RESULTS ON THE TEST SET	23
4.2	MODEL PERFORMANCE COMPARISON	24

LIST OF FIGURES

FIGURE	TITLE	PAGE NO
1.1	BINARY-TO-IMAGE REPRESENTATION	2
1.2	PROPOSED MALWARE CLASSIFICATION WORKFLOW	3
3.1	MOBILEVIT–TINYVIT ENSEMBLE ARCHITECTURE	12
3.2	USE CASE DIAGRAM	15
3.3	ACTIVITY DIAGRAM	16
3.4	SEQUENCE DIAGRAM	17
4.1	DATASET PREPARATION FLOW	19

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Malware, or malicious software, refers to programs designed to perform unauthorized or harmful activities on computer systems. Such activities may include stealing sensitive information, modifying files, disrupting services, gaining unauthorized access, or exploiting system resources.

Common categories of malware include viruses, worms, Trojans, ransomware, spyware, and other malicious programs. As malware becomes more sophisticated, detecting and classifying new variants has become an important cybersecurity challenge.

Traditional antivirus systems frequently use signature-based detection. In this approach, known malicious patterns are stored in a database and compared against new files. Although signature-based detection can be effective for known malware, it may not recognize variants that have been modified through obfuscation, packing, or other transformations.

Machine learning provides an alternative approach by learning patterns from labeled examples. Deep learning methods can automatically learn useful representations from complex data, reducing dependence on manually selected features.

One approach to malware classification is to visualize binary data as grayscale images [1]. A binary file consists of bytes, each having a value between 0 and 255. These values can be interpreted as grayscale pixel intensities and arranged into a two-dimensional image.

For example:

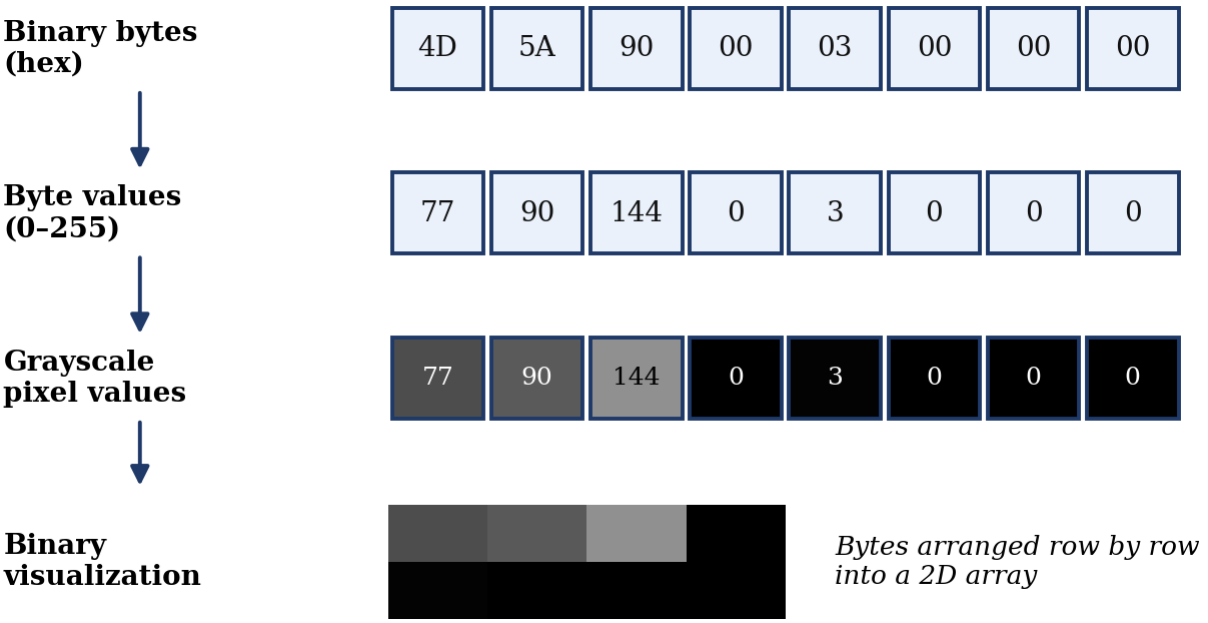


Figure 1.1 Binary-to-Image Representation

The resulting images may contain patterns associated with the underlying file structure and byte distribution. Image-classification models can learn to distinguish patterns associated with different malware families.

Convolutional Neural Networks (CNNs) have been used for image-based malware classification. However, Vision Transformers [2] offer an alternative approach by using attention mechanisms to model relationships between image regions.

Large deep learning models may require considerable memory and computational resources. Lightweight architectures are therefore useful to investigate when computational efficiency is an important design consideration.

This project studies two lightweight Vision Transformer architectures:

1. **MobileViT [3]:** Combines convolutional operations with Transformer-based global representation learning.
2. **TinyViT [4]:** Provides a compact Vision Transformer architecture intended to reduce computational requirements.

The proposed ensemble combines their predictions to investigate whether the models can complement one another.

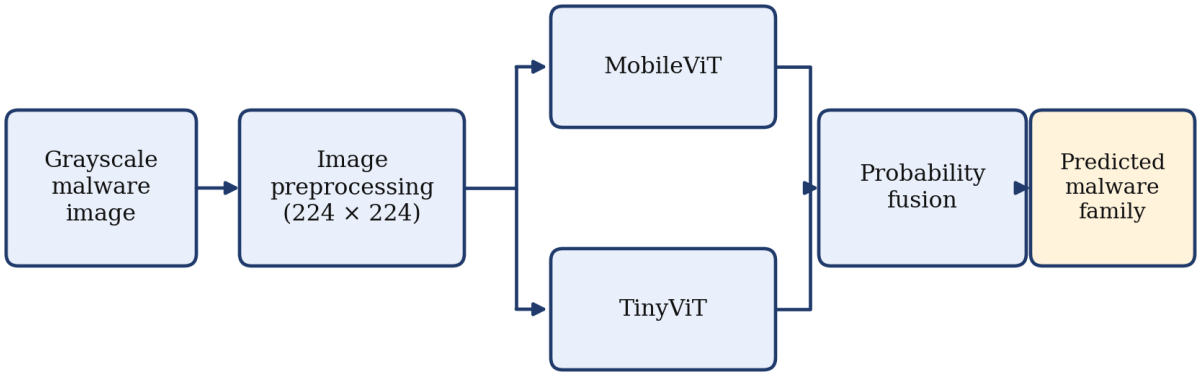


Figure 1.2 Proposed Malware Classification Workflow

1.2 PROBLEM STATEMENT

Traditional signature-based malware detection depends heavily on known signatures and may face difficulties identifying previously unseen or modified malware variants.

Deep learning-based malware classification can automatically learn patterns from data. However, the effectiveness of a model must be considered alongside its computational cost, memory requirements, and ability to classify different malware families.

A single model may misclassify certain families because of class imbalance, similar visual patterns, or insufficient training examples. Combining multiple models may improve classification performance, but it can also increase inference time and model size.

Therefore, the problem addressed by this project is:

> ***To develop and evaluate an ensemble of lightweight Vision Transformers, MobileViT and TinyViT, for malware-family classification using image-based representations, while measuring classification performance and computational efficiency.***

The current experiment focuses on 25 malware-family classes. It does not yet establish benign-versus-malware detection because the current classification checkpoint does not include a benign class.

1.3 OBJECTIVES

The main objectives are:

1. To study existing malware detection and classification techniques.

2. To investigate malware visualization using grayscale images.
3. To prepare and preprocess a labeled malware image dataset.
4. To implement MobileViT for malware-family classification.
5. To implement TinyViT using a compatible dataset and documented training configuration.
6. To develop an ensemble mechanism that combines the outputs of both models.
7. To evaluate the individual models and ensemble using accuracy, precision, recall, and F1-score.
8. To measure model size, parameter count, and inference time.
9. To investigate class imbalance and per-family classification errors.
10. To develop a prototype interface for demonstrating the classification workflow.
11. To investigate the limitations of image-based classification when applied to arbitrary executable files.

1.4 SCOPE

The project includes:

- Malware image dataset preparation.
- Image preprocessing and normalization.
- MobileViT model implementation.
- TinyViT model implementation.
- Probability-level ensemble fusion.
- Malware-family classification.
- Performance evaluation.
- Computational-efficiency analysis.
- Prototype web-interface development.

The initial experiments use an established malware image dataset. The current MobileViT checkpoint supports 25 malware families.

The project does not claim that every executable file can be classified correctly or that the system can replace commercial antivirus software. Generalization to new malware families, benign files, and arbitrary executable formats requires additional experiments.

The binary-to-image conversion prototype must also be validated against the dataset's original representation before the dataset-classification results can be treated as evidence of end-to-end executable classification.

1.5 SECURITY REQUIREMENTS AND SECURITY CONSIDERATIONS

Since malware analysis involves potentially unsafe files, security must be considered throughout the system.

1. Safe file handling

   All uploaded files must be treated as untrusted input.

2. No execution of uploaded files

   The application should read file contents without executing the uploaded program.

3. File validation

   The application should validate file size, format, and basic file properties before processing.

4. Upload size restriction

   A maximum file size should be imposed to prevent excessive memory usage and processing time.

5. Temporary-file management

   Temporary files should be stored in a controlled directory and removed after processing.

6. Filename sanitization

   The application should not use an uploaded filename directly as a filesystem path.

7. Model protection

   Model checkpoints should be stored separately from uploaded files and protected against unauthorized modification.

8. Resource management

   Processing time, concurrent requests, and memory usage should be limited.

9. Dataset integrity

   The dataset should be checked for duplicate samples and potential overlap between training and testing data.

10. Error management

    The application should display understandable error messages without exposing internal paths or debugging information.

1.6 SDG ALIGNMENT

SDG 9 Industry, Innovation and Infrastructure

The project investigates an AI-based cybersecurity approach intended to support more resilient digital infrastructure.

SDG 16 Peace, Justice and Strong Institutions

The project contributes to research into automated security analysis and the protection of digital systems.

SDG 8 Decent Work and Economic Growth

Improved cybersecurity techniques can support safer and more reliable digital services.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEMS WITH SECURITY APPROACHES

2.1.1 SIGNATURE-BASED MALWARE DETECTION

Signature-based detection identifies malware by comparing files against known malicious patterns.

Workflow:

File → Signature extraction or matching → Signature database → Detection result

Advantages:

- Fast detection for known threats.
- Established in traditional antivirus systems.
- Relatively straightforward to interpret.

Limitations:

- Depends on available signatures.
- May not identify previously unseen malware.
- Modified malware may evade existing signatures.

2.1.2 STATIC MALWARE ANALYSIS

Static analysis examines file contents without executing the program.

Possible features include:

- Byte sequences.
- File headers.
- Imported functions.
- Strings.
- Sections and structural information.
- Entropy and byte distributions.

Static analysis reduces the need to execute potentially malicious programs. However, packing, obfuscation, and other transformations may make interpretation difficult.

2.1.3 MACHINE LEARNING-BASED DETECTION

Machine learning-based systems learn patterns from labeled benign and malicious samples or from malware-family datasets.

The general workflow is:

Dataset → Feature extraction → Model training → Classification → Evaluation

The effectiveness of these systems depends on the quality of the training data, selected features, model architecture, and evaluation procedure.

2.1.4 CNN-BASED MALWARE IMAGE CLASSIFICATION

CNN-based methods treat malware visualizations as images. Convolutional layers learn spatial features, and a classification layer predicts the class.

Advantages include automatic feature extraction and the ability to process visual patterns.

However, the performance of a CNN depends on the dataset and architecture, and its ability to model long-range relationships may differ from that of attention-based models.

2.1.5 VISION TRANSFORMER-BASED CLASSIFICATION

Vision Transformers use attention mechanisms to model relationships between image regions. A typical workflow is:

Image → Patch representation → Transformer layers → Feature representation → Classification

MobileViT combines convolutional operations and Transformer processing. TinyViT provides another compact architecture for vision tasks.

The proposed project evaluates these two models individually and then investigates whether combining their outputs improves malware-family classification.

2.2 RESEARCH GAPS

The project investigates the following research gaps:

1. The relationship between classification performance and computational efficiency needs to be evaluated.
2. Lightweight Transformer architectures should be compared under consistent experimental conditions.
3. Class imbalance may cause aggregate accuracy to conceal poor performance on smaller classes.
4. Different models may make different errors, creating an opportunity for ensemble learning.
5. Ensembles can increase inference time, so any performance improvement must be compared against additional computational cost.
6. Dataset-based image classification does not automatically validate an independently implemented executable-to-image pipeline.
7. Generalization to benign files and previously unseen malware families requires separate validation.

The main research question is:

> ***Can an ensemble of MobileViT and TinyViT provide effective malware-family classification while maintaining acceptable computational efficiency?***

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system consists of dataset preparation, preprocessing, two model branches, ensemble fusion, and final classification.

The two models receive the same labeled image after compatible preprocessing. Each model generates a probability for every malware family. These probability vectors are combined using a weighted average.

The class with the highest fused probability becomes the final prediction.

Proposed workflow:

Input image → Preprocessing → MobileViT prediction
Input image → Preprocessing → TinyViT prediction
MobileViT probabilities + TinyViT probabilities → Ensemble fusion → Predicted malware family

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

- Learn image representations using two lightweight Vision Transformers.
- Compare the performance of MobileViT and TinyViT.
- Combine their predictions through a reproducible fusion method.
- Evaluate the ensemble on held-out test data.
- Measure classification performance and computational efficiency.
- Provide a foundation for a secure analysis interface.

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

Automatic feature learning: The models learn image features from the training dataset.

Model comparison: Two lightweight Transformer architectures can be compared using consistent data splits.

Potential complementary predictions: The models may make different errors, allowing ensemble fusion to improve classification.

Efficiency analysis: Parameter count, model size, and inference time can be measured.

Extensibility: The architecture can later be evaluated with additional models and datasets.

2.3.4 LIMITATIONS OF THE PROPOSED SYSTEM

- Performance depends on dataset quality and representativeness.
- Imbalanced classes may reduce minority-family performance.
- Running two models can increase inference cost.
- High confidence does not guarantee a correct prediction.
- The current 25-class model does not classify benign files.
- Arbitrary executable classification requires validation of the binary-to-image representation and suitable training data.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The proposed architecture contains the following modules:

1. Dataset management.
2. Image preprocessing.
3. MobileViT classification.
4. TinyViT classification.
5. Ensemble probability fusion.
6. Performance evaluation.
7. Optional web interface.

Architecture Description

The dataset provides labeled grayscale malware images. The preprocessing module converts the images into the input format required by the models.

The preprocessed image is supplied to both MobileViT and TinyViT. Each model returns a probability vector corresponding to the malware-family classes.

The ensemble module combines these probability vectors. The final class is selected from the combined output.

The evaluation module calculates classification metrics and computational measurements.

The web interface, when implemented and validated, can provide a way to submit an approved input and display the model's prediction.

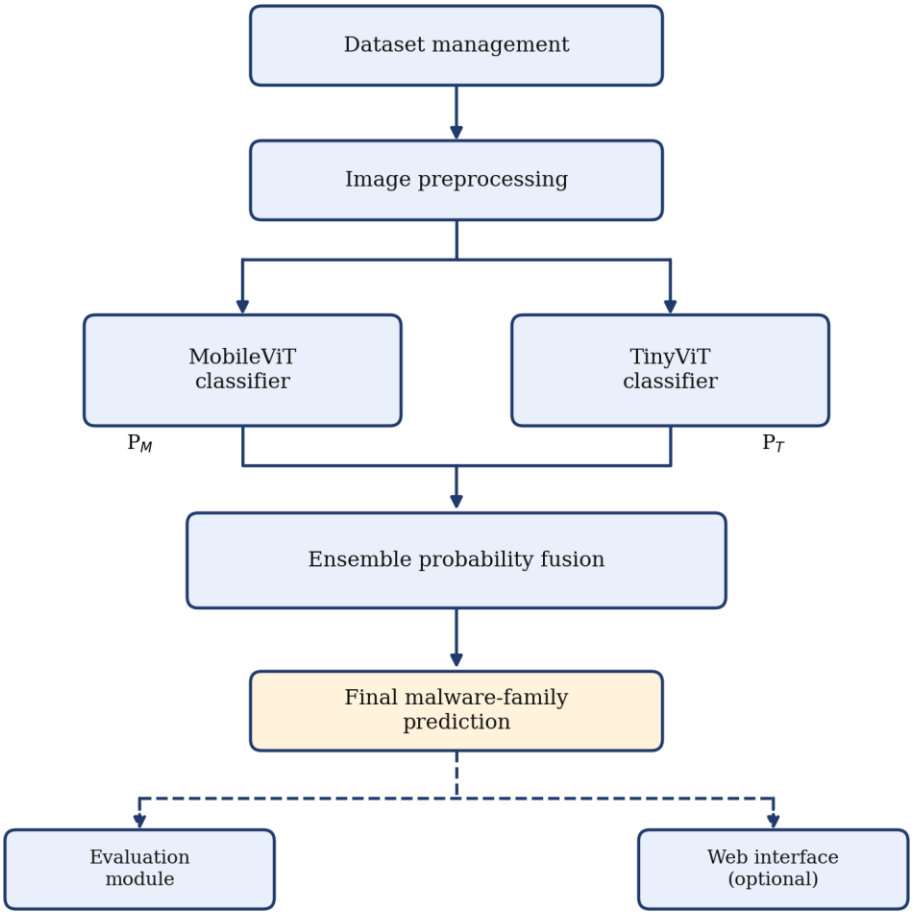


Figure 3.1 MobileViT–TinyViT Ensemble Architecture

3.2 MODULE DESIGN

Module 1 Dataset Management

Responsibilities:

- Load labeled malware images.
- Organize images by malware family.
- Maintain class labels and class indices.
- Prepare training, validation, and testing splits.
- Record image counts.
- Check for possible duplicate samples.

The test set must remain separate from model training and model-selection decisions.

Module 2 Image Preprocessing

Responsibilities:

- Load grayscale images.
- Convert images to the required number of channels.
- Resize images to the model's input dimensions.
- Convert images to tensors.
- Normalize pixel values.

The current MobileViT inference pipeline uses a 224 × 224 input size.

Module 3 MobileViT Classifier

MobileViT is the first model in the proposed system.

It combines local convolutional processing with Transformer-based representation learning. Convolutional operations capture local spatial patterns, while attention mechanisms model relationships between image regions.

The current implementation uses a trained checkpoint containing the model state dictionary and 25 class labels.

Module 4 TinyViT Classifier

TinyViT is the second model.

Its role is to independently learn image features and classify malware-family images. The exact model variant, training settings, preprocessing, and checkpoint must be documented when implementation is completed.

TinyViT should be evaluated independently before ensemble fusion is assessed.

Module 5 Ensemble Fusion

The ensemble module combines the probability vectors produced by MobileViT and TinyViT.

Let:

- P_{M} represent the MobileViT probability vector.
- P_{T} represent the TinyViT probability vector.
- α represent the MobileViT fusion weight.

The ensemble probability is:

$$P_E = \alpha P_M + (1 - \alpha) P_T$$

where $0 \leq \alpha \leq 1$.

If $\alpha = 0.5$, both models contribute equally. Other values may be investigated using validation data.

The final predicted class is:

$$\hat{y} = \text{argmax}(P_E)$$

The fusion weight must be selected using validation data rather than the final test set.

Module 6 — Evaluation

The evaluation module measures:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Macro and weighted averages.
- Confusion matrix.
- Parameter count.
- Model size.
- Inference time.

All models should use the same held-out test set and class mapping.

Module 7 Web Application

A Flask-based interface is a planned demonstration component.

Its responsibilities may include:

- Accepting an input file.
- Validating file size and format.
- Performing preprocessing.
- Running model inference.
- Displaying the predicted malware family.

- Displaying the prediction score.
- Handling invalid inputs safely.

The interface should not describe a sample as benign unless a suitable benign-class model has been trained and evaluated.

3.3 DATA FLOW AND UML DIAGRAMS

3.3.1 USE CASE DIAGRAM

Actor: User or security analyst.

Use cases:

- Select an authorized input.
- Request analysis.
- Run model prediction.
- View the predicted malware family.
- View evaluation results.

The user interacts with the application, while preprocessing and model inference take place inside the analysis system.

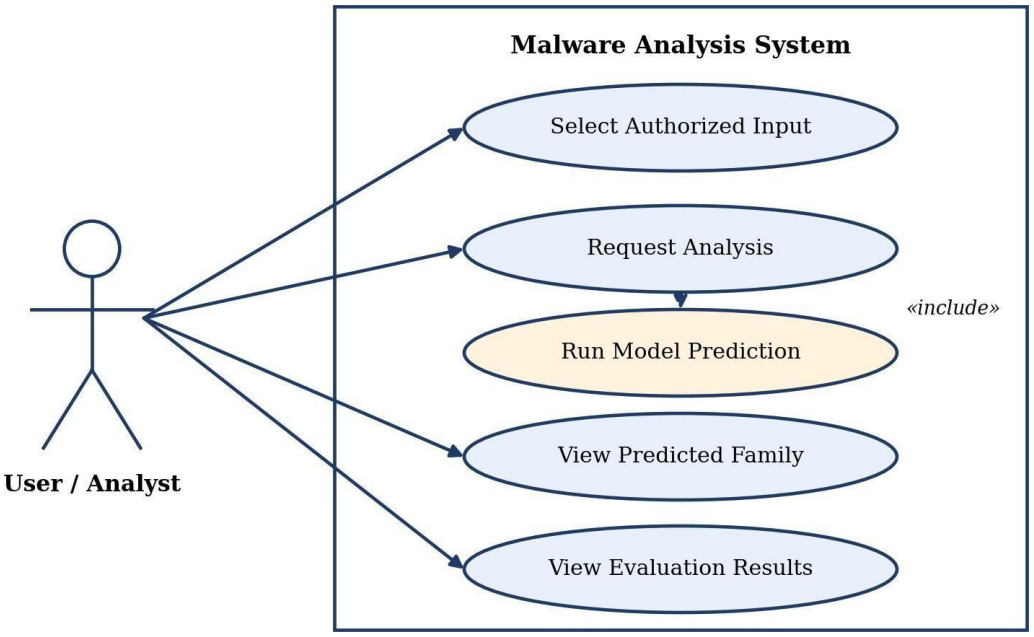


Figure 3.2 Use Case Diagram

3.3.2 ACTIVITY DIAGRAM

The activity flow is:

Start → Validate input → Preprocess image → Run MobileViT → Run TinyViT → Fuse probabilities → Display prediction → End.

If the input is invalid, the application should display an error and stop processing.

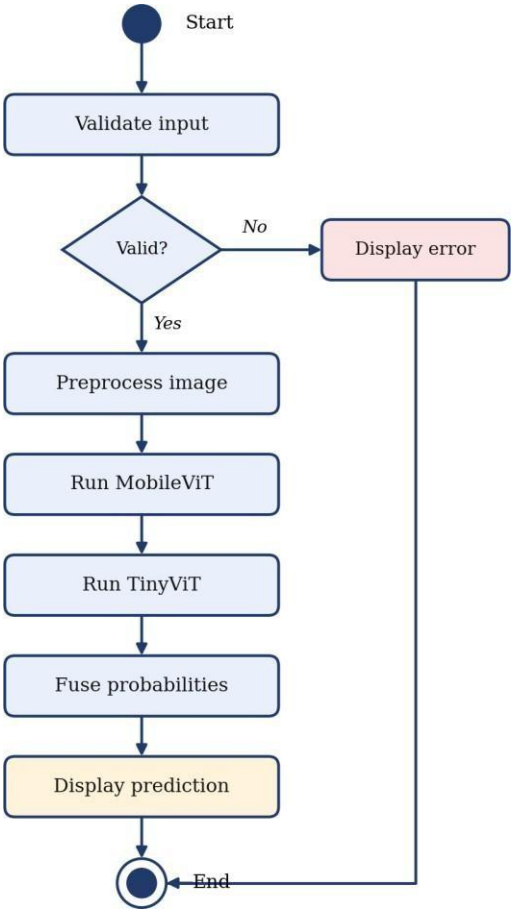


Figure 3.3 Activity Diagram

3.3.3 SEQUENCE DIAGRAM

The sequence of operations is:

1. The user submits an input.
2. The application validates it.
3. The preprocessing module prepares the input.
4. MobileViT generates its probability vector.
5. TinyViT generates its probability vector.

6. The ensemble module combines the vectors.
7. The application displays the final prediction.

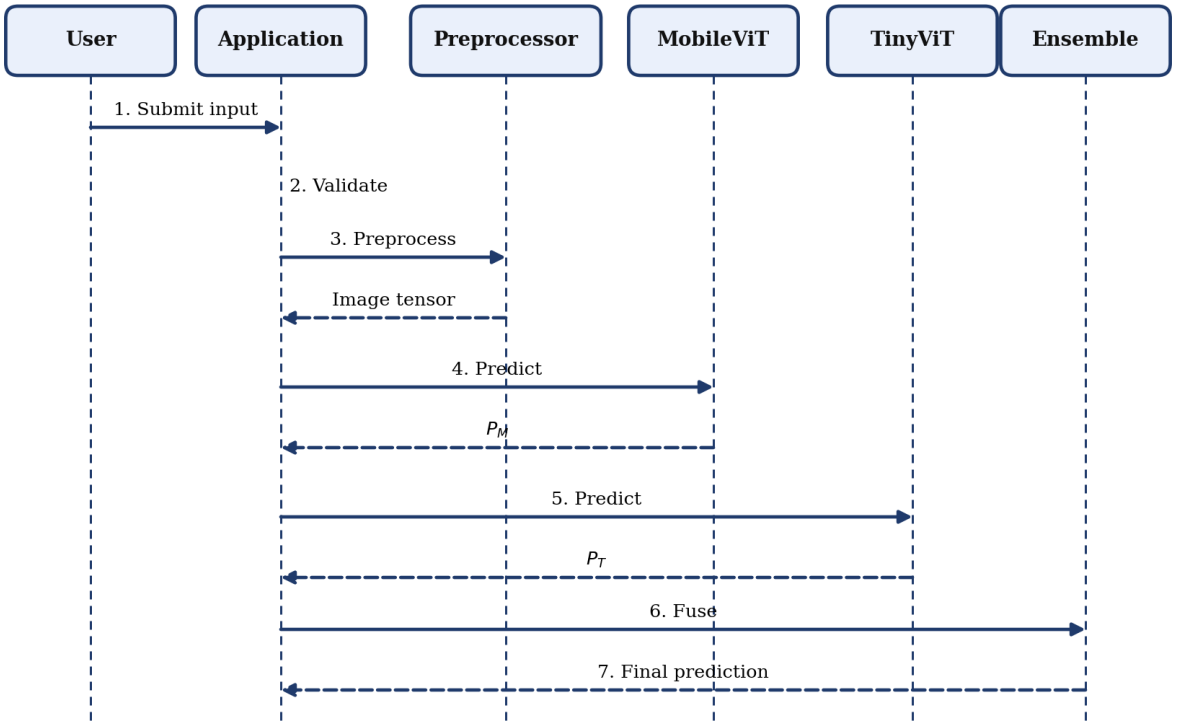


Figure 3.4 Sequence Diagram

3.3.4 DATA FLOW DIAGRAM

Input image → Preprocessing module → Individual models → Ensemble module → Prediction output.

The evaluation module compares predictions against ground-truth labels to calculate performance metrics.

3.4 THREAT MODEL AND ATTACK SURFACE

The system may encounter several threats.

Malicious upload: An uploaded file may be malformed or designed to trigger unexpected processing behaviour.

Oversized input: A large file may consume excessive memory or processing time.

Unsafe filename: An unsanitized filename may cause unintended filesystem access.

Denial of service: Repeated requests may exhaust system resources.

Checkpoint tampering: Unauthorized modification of model files may affect predictions.

Dataset leakage: Duplicate or related samples across training and test splits may produce overly optimistic metrics.

Adversarial inputs: Modified samples may cause incorrect model predictions.

3.5 SECURITY CONTROLS

Table 3.1 Security Controls Against Identified Threats

Threat	Security Control
Malformed input	Validate input before processing
Oversized file	Enforce maximum upload size
Unsafe filename	Generate internal filenames
Execution risk	Never execute uploaded files
Temporary-file abuse	Use controlled temporary storage and cleanup
Resource exhaustion	Limit request size and processing time
Model modification	Restrict checkpoint write access
Dataset leakage	Check duplicates and maintain separate splits
Information leakage	Display controlled error messages

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The current image-classification experiment uses the Malimg malware image dataset [1]. The dataset contains grayscale images organized by malware-family labels.

The saved MobileViT checkpoint contains 25 classes, and the current test evaluation includes 957 images.

The dataset is divided into training, validation, and testing subsets. Training data are used to learn model parameters, validation data are used for model selection and fusion-weight selection, and test data are used for final evaluation.

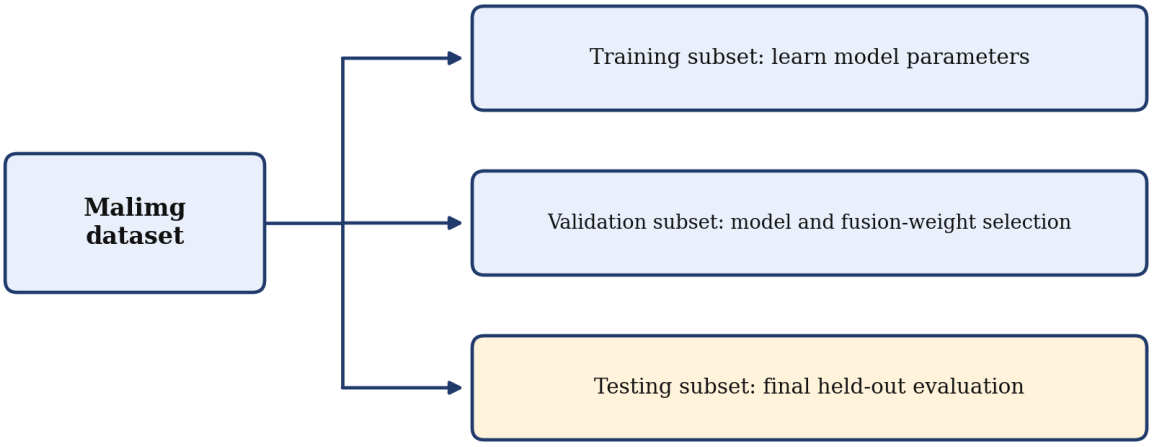


Figure 4.1 Dataset Preparation Flow

The exact sample counts for all three subsets should be obtained from the project's actual dataset folders and training logs.

Dataset considerations

The following issues must be considered:

- Number of images in each family.
- Class imbalance.
- Duplicate or highly similar samples.
- Separation between training and testing data.
- Correct mapping between class names and class indices.

- Generalization to samples outside the training distribution.

Dataset limitation

The current experiment is based on malware images. It does not, by itself, demonstrate that arbitrary executable files can be converted into equivalent images and classified correctly.

4.2 DATA PREPROCESSING

The current MobileViT preprocessing pipeline is:

1. Load the grayscale image.
2. Convert it to three channels.
3. Resize it to 224 × 224.
4. Convert it to a tensor.
5. Normalize the tensor.
6. Pass the tensor to the trained model.

The normalization used by the current MobileViT inference pipeline is:

Mean:

$$\mu = [0.485, 0.456, 0.406]$$

Standard deviation:

$$\sigma = [0.229, 0.224, 0.225]$$

TinyViT preprocessing must match its training configuration. If its input normalization differs, the difference must be documented and applied consistently.

Binary-to-image conversion prototype

A separate preprocessing script reads bytes from a file and interprets each byte as a grayscale intensity.

Each byte value falls within the range 0–255. These values are arranged row by row into a two-dimensional array and saved as an image.

The current prototype uses a width of 64 pixels. However, matching the width of the Malimg images does not prove that the same original conversion method has been reproduced.

Therefore, the converter is treated as a prototype until the original dataset-generation method is verified.

4.2.1 ALGORITHMS AND METHODOLOGY

Algorithm 1 — Image Preprocessing

Input: Labeled grayscale image.

1. Load the image from the selected dataset split.
2. Convert it to the required channel format.
3. Resize it to 224 × 224.
4. Convert pixel values to a tensor.
5. Apply the model's normalization.
6. Return the tensor and ground-truth label.

Output: Preprocessed image tensor and class label.

Algorithm 2 — MobileViT Classification

Input: Preprocessed image tensor.

1. Load the trusted MobileViT checkpoint.
2. Restore the model parameters.
3. Load the ordered class list.
4. Set the model to evaluation mode.
5. Perform inference without gradient calculation.
6. Calculate class probabilities using softmax.
7. Identify the class with the highest probability.
8. Return the predicted family and prediction score.

Output: Predicted malware family and probability score.

Algorithm 3 — TinyViT Classification

Input: Preprocessed image tensor.

1. Load the trained TinyViT checkpoint.
2. Restore the model parameters.
3. Set the model to evaluation mode.
4. Perform inference.
5. Calculate class probabilities.

6. Identify the highest-probability class.
7. Return the predicted class and probabilities.

Output: TinyViT prediction and probability vector.

This algorithm describes the planned TinyViT component; its implementation and results must be verified.

Algorithm 4 — Ensemble Classification

Input: MobileViT and TinyViT probability vectors.

1. Obtain the MobileViT probability vector.
2. Obtain the TinyViT probability vector.
3. Verify that both vectors use the same ordered class mapping.
4. Apply the selected fusion weight.
5. Combine the probability vectors using the weighted-average equation.
6. Select the class with the highest combined probability.
7. Return the ensemble prediction.

Output: Final predicted malware family.

Training methodology

The intended training methodology is:

Dataset preparation → Training and validation split → Preprocessing → MobileViT training → TinyViT training → Validation → Fusion-weight selection → Held-out test evaluation.

The models should be trained independently, and the ensemble weight should be selected using validation data. The final test set should not be used to tune model parameters.

4.2.2 RESULTS AND DISCUSSION

Current MobileViT evaluation

The existing MobileViT checkpoint was evaluated on 957 test images belonging to 25 malware families.

The measured accuracy was **98.54%**.

The classification report provided the following aggregate metrics:

Table 4.1 – MobileViT Evaluation Results on the Test Set

Metric	MobileViT result
Test images	957
Malware-family classes	25
Accuracy	98.54%
Macro precision	0.95
Macro recall	0.95
Macro F1-score	0.95
Weighted precision	0.97
Weighted recall	0.99
Weighted F1-score	0.98

These are the current MobileViT results, not results for TinyViT or the ensemble.

Class-wise analysis

The current test set is imbalanced. For example, the reported support includes:

- Allaple.A: 296 images.
- Allaple.L: 160 images.
- Yuner.A: 80 images.
- Several families with fewer than 20 images.

The current classification report showed zero precision, recall, and F1-score for Autorun.K. The F1-scores for Swizzor.gen!E, Swizzor.gen!I, and Yuner.A were approximately 0.93.

These observations indicate that aggregate accuracy alone does not fully describe performance. Macro metrics and class-wise results are important when evaluating imbalanced datasets.

Single-image prediction

The existing prediction script was tested with an image from the Agent.FYI test folder. It predicted Agent.FYI with a displayed confidence of 99.97%.

This is an example prediction, not an additional dataset-level performance measurement.

TinyViT evaluation

TinyViT must be trained and evaluated before its performance can be reported. The same test set and compatible class mapping should be used for a fair comparison.

Ensemble evaluation

The ensemble must be evaluated after probability fusion has been implemented. Its performance may be better than, similar to, or worse than the individual models.

The final report should compare:

Table 4.2 Model Performance Comparison

Metric	MobileViT	TinyViT	Ensemble
Accuracy	98.54%	Pending	Pending
Precision	0.95 macro	Pending	Pending
Recall	0.95 macro	Pending	Pending
F1-score	0.95 macro	Pending	Pending
Parameter count	Measure	Pending	Measure combined cost
Model size	Measure	Pending	Measure combined cost
Inference time	Measure	Pending	Pending

The pending fields must be replaced with measured results before the final report is submitted.

Computational efficiency

The models should be compared using the same hardware and a consistent inference procedure.

The following measurements should be collected:

- Number of trainable parameters.
- Size of the saved checkpoint.
- Inference time per image.
- Batch size.
- Device used for evaluation.
- FLOPs, if a reliable measurement method is available.

An ensemble generally requires running both models, so its additional computational cost must be considered alongside any change in classification performance.

Discussion

The current MobileViT result establishes a useful baseline for the project. However, the presence of class imbalance and weak performance on at least one class means that the result should not be interpreted as uniformly high performance across all malware families.

The final comparison with TinyViT and the ensemble will determine whether the combination provides measurable benefits.

The current experiment is a 25-class malware-family classification task. It does not establish benign-versus-malware detection or prove that arbitrary executable files can be classified correctly through the custom binary-to-image pipeline.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This project investigates an ensemble of lightweight Vision Transformers for malware-family classification using grayscale malware images. The proposed approach uses MobileViT and TinyViT as individual classifiers and combines their probability outputs to generate a final prediction.

The current MobileViT model was evaluated on 957 test images across 25 malware families and achieved 98.54% accuracy. The reported macro precision, recall, and F1-score were 0.95, 0.95, and 0.95, respectively.

These results provide the initial baseline for evaluating TinyViT and the ensemble. The effectiveness of the ensemble must be established through additional experiments using the same held-out test set and consistent evaluation conditions.

The project also investigates computational efficiency through parameter count, model size, and inference time. These measurements are important because an ensemble may improve prediction performance while increasing computational cost.

The current work focuses on malware-family classification. Additional data and training are required to support benign-versus-malware detection. The custom binary-to-image pipeline must also be validated against the original dataset-generation method before making claims about end-to-end executable classification.

5.2 FUTURE WORK

The following extensions are planned:

1. Complete TinyViT implementation and training.
2. Implement MobileViT–TinyViT probability fusion.
3. Select the ensemble weight using validation data.
4. Evaluate all three approaches on the same held-out test set.
5. Compare classification metrics and computational efficiency.
6. Investigate class imbalance and difficult malware families.
7. Verify the binary-to-image conversion methodology.

8. Add a suitable benign dataset and retrain if binary detection is required.
9. Complete and test the Flask interface.
10. Evaluate generalization using additional datasets or samples not represented in training.

REFERENCES

[1] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, “Malware images: Visualization and automatic classification,” *Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec)*, 2011.

[2] A. Dosovitskiy et al., “An image is worth 16×16 words: Transformers for image recognition at scale,” *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.11929.

[3] S. Mehta and M. Rastegari, “MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer,” *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2110.02178.

[4] K. Wu et al., “TinyViT: Fast pretraining distillation for small vision transformers,” *European Conference on Computer Vision (ECCV)*, 2022. arXiv:2207.10666.

[5] A. Vaswani et al., “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.

Verify the complete bibliographic details against the original publications before final submission.

APPENDIX

Appendix A — Project files

The current project includes the following components:

- models/mobilevit.py — MobileViT model creation.
- scripts/train.py — MobileViT training.
- scripts/evaluate.py — evaluation on the held-out test set.
- scripts/predict.py — single-image family prediction.
- preprocessing/binary_to_image.py — binary-to-grayscale conversion prototype.
- scripts/binary_predict.py — prototype binary-to-image and MobileViT prediction workflow.

Planned components include TinyViT training and evaluation scripts, ensemble fusion, and the Flask application.

Appendix B — Dataset classes

The saved MobileViT checkpoint contains 25 malware-family labels. The final appendix should list the exact class names in the order stored in the checkpoint.

The same class mapping must be used when interpreting TinyViT and ensemble predictions.

Appendix C — Training configuration

Parameter	Value
Dataset	Malimg image dataset
Number of test images	957
Number of classes	25
Model input size	224 × 224
Implemented model	MobileViT
Checkpoint fields	model_state_dict, classes
Test accuracy	98.54%
TinyViT architecture/configuration	To be recorded
Ensemble weight	To be selected on validation data
Training epochs	Copy from actual training log
Batch size	Copy from training script
Optimizer	Copy from training script
Learning rate	Copy from training script
Evaluation device	Record actual device

Appendix D — Screenshots to include

1. Project folder structure.
2. Dataset folder structure.
3. MobileViT training output.
4. MobileViT evaluation output.
5. Single-image prediction output.
6. Binary-to-image conversion output.
7. TinyViT training output after implementation.
8. Ensemble prediction output after implementation.
9. Classification reports and confusion matrices.
10. Flask interface and result page after implementation.

Appendix E — Recommended final diagrams

- Figure 1.1: Binary-to-image representation.
- Figure 1.2: Proposed malware classification workflow.

- Figure 3.1: MobileViT–TinyViT ensemble architecture.
- Figure 3.2: Use case diagram.
- Figure 3.3: Activity diagram.
- Figure 3.4: Sequence diagram.
- Figure 4.1: Dataset preparation workflow.
- Figure 4.2: Individual-model and ensemble comparison chart.